

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
Design and Analysis of Algorithms (DAA)

CASE STUDY REPORT

Airline Reservation Sorting System using Merge Sort

Student Name	Paleru Dharani Govardhan
Register Number	192525280
Subject	Design and Analysis of Algorithms (DAA)
Case Study	Merge Sort
Academic Year	2025 - 2029
Year	B.Tech II Year

CERTIFICATE

This is to certify that this case study report titled "**Airline Reservation Sorting System using Merge Sort**" is a bona fide record of work carried out by **Paleru Dharani Govardhan** (Register Number: **192525280**), submitted in partial fulfilment of the requirements for the course **Design and Analysis of Algorithms (DAA)**, during the academic year 2025-2026.

The report has been prepared under the guidance of the course faculty and is found to be satisfactory in terms of technical content, originality, and presentation.

Date: _____

Place: _____

Signature of Faculty-in-Charge

TABLE OF CONTENTS

1. Cover Page.....	1
2. Certificate	2
3. Table of Contents	3
4. Title.....	5
5. Description of the Real World Problem	5
6. Problem Statement	5
7. Objectives.....	5
8. Existing System	5
9. Problems in Existing System.....	6
10. Proposed System.....	6
11. Algorithm Used (Merge Sort)	6
12. Why Merge Sort?	6
13. Working Principle of Merge Sort.....	7
14. Divide and Conquer Strategy	7
15. Flow Diagram of Merge Sort	7
16. Algorithm (Pseudocode).....	8
17. Python Program	8
18. Explanation of the Code	10
19. Manual Execution (Dry Run)	10
20. Complexity Analysis.....	13
21. Stability Analysis.....	13
22. Space Complexity	13
23. Applications.....	14
24. Advantages.....	14
25. Limitations.....	14
26. Comparison with Other Sorting Algorithms	15
27. Real World Implementation in Airline Reservation Systems	15
28. Sample Input	16
29. Sample Output	16
30. Screenshot Placeholder for Python Program	17
31. Screenshot Placeholder for Output.....	17
32. Conclusion	17
33. References.....	17

Case Study: Airline Reservation Sorting System using Merge Sort

5. Description of the Real World Problem

Modern airline companies handle **thousands of flight bookings every day** across multiple cities, routes, and time zones. Each booking record carries important attributes such as the passenger's name, the flight number, the scheduled departure time, and a priority level that reflects factors such as ticket class, loyalty status, or special assistance requirements.

Before this data can be usefully displayed to airline staff, boarding gate officers, or passengers themselves, it must be **sorted in a meaningful order** — typically by departure time, and within the same departure time, by passenger priority. Because reservation systems must serve accurate, ordered information under continuous real-time updates (new bookings, cancellations, and rescheduling), the underlying sorting mechanism must be **fast, predictable, and stable**.

This case study examines how the **Merge Sort algorithm**, a classic Divide and Conquer technique, can be applied to build an efficient and stable sorting engine for an airline reservation system.

6. Problem Statement

Design an algorithm that can sort a large collection of airline booking records primarily by **departure time** and secondarily by **passenger priority**, such that:

- Records with the same departure time retain their relative priority order (**stability**).
- The algorithm performs efficiently even when the number of bookings is very large.
- The solution can accommodate frequent insertions arising from real-time booking updates.

7. Objectives

- To analyse the working of Merge Sort using the Divide and Conquer strategy.
- To design a sorting solution for airline booking records based on departure time and priority.
- To prove and demonstrate the **stability** of Merge Sort through a manual dry run.
- To evaluate the **time and space complexity** of Merge Sort in Best, Average, and Worst cases.
- To compare Merge Sort with other standard sorting algorithms.
- To justify why Merge Sort is the most suitable algorithm for airline reservation sorting.

8. Existing System

Many legacy airline reservation systems rely on **simple comparison-based sorting techniques** such as Bubble Sort, Selection Sort, or Insertion Sort to arrange booking records. These techniques were adequate when the number of daily bookings was small, and computing resources were limited.

In such systems, booking records are usually stored in flat arrays or simple linked lists, and sorting is triggered on demand — for example, whenever a staff member requests the boarding list for a flight.

9. Problems in Existing System

- **Large volume of booking records:** With thousands of passengers per day across hundreds of flights, simple quadratic-time algorithms ($O(n^2)$) become extremely slow.
- **Real-time booking updates:** Every new booking, cancellation, or modification may require re-sorting, and inefficient algorithms cannot keep up with frequent updates.
- **Lack of stability:** Some algorithms (e.g., Selection Sort, Quick Sort in typical implementations) do not preserve the relative order of equal-priority passengers, causing unfair reordering.
- **Poor scalability:** As airline networks expand, the existing sorting mechanism does not scale well to handle peak-season traffic.
- **High latency:** Passengers and staff experience delays when boarding lists or itinerary displays take too long to refresh.

10. Proposed System

The proposed system replaces inefficient sorting mechanisms with the **Merge Sort algorithm**, which guarantees $O(n \log n)$ time complexity in all cases (best, average, and worst) and preserves the relative order of records with equal keys (stability).

The proposed system sorts booking records using a **composite key**: departure time as the primary key and priority as the secondary key. This ensures that flights depart in correct chronological order, and within each flight, passengers are correctly prioritised.

11. Algorithm Used (Merge Sort)

Merge Sort is a comparison-based, Divide and Conquer sorting algorithm invented by John von Neumann in 1945. It divides the input array into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted array.

12. Why Merge Sort?

- Guarantees **$O(n \log n)$** performance regardless of the initial order of data.
- **Stable** sorting — critical for preserving passenger priority order.

- Performs predictably well on **large datasets**, unlike quadratic algorithms.
- Well suited to **external sorting** when booking data does not fit entirely in memory.
- Naturally parallelisable, allowing sorting workloads to be distributed across servers.

13. Working Principle of Merge Sort

Merge Sort operates in two clear phases:

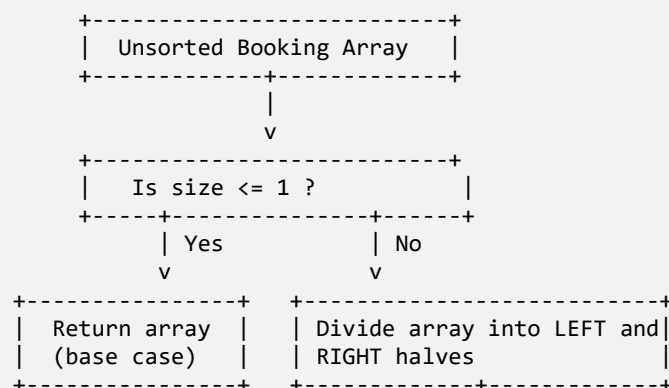
1. **Divide** — The unsorted array of booking records is repeatedly split into two halves until each sub-array contains only a single record (which is trivially sorted).
2. **Combine (Merge)** — Adjacent sorted sub-arrays are merged together, comparing the front elements of each sub-array and placing the smaller (or equal, taken from the left to preserve order) element into the output array.

14. Divide and Conquer Strategy

The **Divide and Conquer** paradigm solves a problem by breaking it into smaller sub-problems of the same type, solving each sub-problem recursively, and combining their solutions to solve the original problem. Merge Sort applies this paradigm as follows:

Step	Action Performed
Divide	Split the array of n booking records into two halves of size n/2 each.
Conquer	Recursively sort each half by applying Merge Sort until the base case (single element) is reached.
Combine	Merge the two sorted halves into one sorted array using the merge procedure.

15. Flow Diagram of Merge Sort



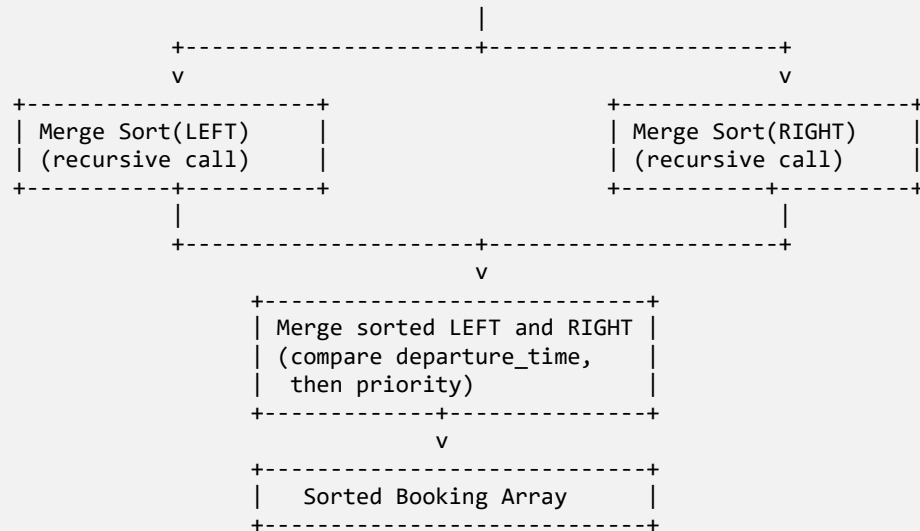


Figure 15.1: Flow diagram of the Merge Sort process for booking records

16. Algorithm (Pseudocode)

```

ALGORITHM MergeSort(A, low, high)
Input:  A[low..high] - array of booking records
Output: A[low..high] - sorted in ascending order of
        (departure_time, priority)

1. IF low < high THEN
2.   mid <- (low + high) / 2
3.   MergeSort(A, low, mid)           // sort left half
4.   MergeSort(A, mid + 1, high)      // sort right half
5.   Merge(A, low, mid, high)        // merge both halves
6. END IF

ALGORITHM Merge(A, low, mid, high)
1. Create temporary arrays L <- A[low..mid]
   R <- A[mid+1..high]
2. i <- 0, j <- 0, k <- low
3. WHILE i < size(L) AND j < size(R) DO
4.   IF L[i].departure_time < R[j].departure_time
   OR (L[i].departure_time = R[j].departure_time
   AND L[i].priority <= R[j].priority) THEN
5.     A[k] <- L[i]; i <- i + 1
6.   ELSE
7.     A[k] <- R[j]; j <- j + 1
8.   END IF
9.   k <- k + 1
10. END WHILE
11. Copy any remaining elements of L into A
12. Copy any remaining elements of R into A
  
```

17. Python Program

```
# -----
```



```

# Airline Reservation Sorting System using Merge Sort
# Sorts booking records by Departure Time (primary key)
# and Priority (secondary key)
# -----

class Booking:
    """Represents a single airline booking record."""

    def __init__(self, name, flight_no, departure_time, priority):
        self.name = name                # Passenger Name
        self.flight_no = flight_no      # Flight Number
        self.departure_time = departure_time # 24-hr format string, e.g. "0930"
        self.priority = priority        # 1 = highest priority

    def __repr__(self):
        return f"{self.name:<15}| {self.flight_no:<6}| " \
               f"{self.departure_time:<6}| P{self.priority}"

def compare(b1, b2):
    """
    Compares two bookings using departure_time as the primary key
    and priority as the secondary key.
    Returns a negative number if b1 should come before b2,
    zero if they are equal, and a positive number otherwise.
    """
    if b1.departure_time != b2.departure_time:
        return -1 if b1.departure_time < b2.departure_time else 1
    if b1.priority != b2.priority:
        return -1 if b1.priority < b2.priority else 1
    return 0

def merge(left, right):
    """
    Merges two already-sorted lists of bookings into a single
    sorted list. Using '<=' (via compare) keeps the merge STABLE:
    when two records are equal, the one from the left half
    (which appeared earlier) is placed first.
    """
    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if compare(left[i], right[j]) <= 0:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    # Append any remaining elements (only one of these has items)
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def merge_sort(bookings):
    """
    Recursively divides the booking list into halves (Divide),

```

```

    sorts each half (Conquer), and merges them back together
    (Combine) until the entire list is sorted.
    """
    if len(bookings) <= 1:
        return bookings          # Base case: already sorted

    mid = len(bookings) // 2
    left_half = merge_sort(bookings[:mid])
    right_half = merge_sort(bookings[mid:])

    return merge(left_half, right_half)

if __name__ == "__main__":
    bookings = [
        Booking("Arjun Rao",      "AI101", "0930", 2),
        Booking("Divya Menon",    "AI202", "0730", 1),
        Booking("Karthik Iyer",   "AI101", "0930", 1),
        Booking("Sneha Reddy",    "AI303", "1100", 3),
        Booking("Vikram Sharma",  "AI202", "0730", 2),
        Booking("Ananya Gupta",   "AI404", "0600", 1),
        Booking("Rahul Verma",    "AI303", "1100", 1),
        Booking("Priya Nair",     "AI404", "0600", 2),
    ]

    print("Before Sorting:")
    for b in bookings:
        print(b)

    sorted_bookings = merge_sort(bookings)

    print("\nAfter Sorting (by Departure Time, then Priority):")
    for b in sorted_bookings:
        print(b)

```

18. Explanation of the Code

- **class Booking** — A simple data container that stores the passenger name, flight number, departure time, and priority for one booking record.
- **compare(b1, b2)** — Implements the composite comparison rule: departure time is checked first; priority is checked only when departure times are equal.
- **merge(left, right)** — Combines two already-sorted lists into one sorted list. Using the '<=' condition (through compare) is what makes the algorithm **stable**.
- **merge_sort(bookings)** — The main recursive function. It repeatedly splits the list in half (Divide), sorts each half recursively (Conquer), and calls merge() to combine the results (Combine).
- **Base case** — When a sub-list has zero or one element, it is already sorted, and recursion stops.
- **Driver code (__main__)** — Creates 8 sample bookings, prints them before sorting, sorts them using merge_sort(), and prints the final sorted list.

19. Manual Execution (Dry Run)

Consider the following **8 unsorted booking records** used as input to the algorithm:

Index	Passenger Name	Flight No.	Departure Time	Priority
0	Arjun Rao	AI101	0930	2
1	Divya Menon	AI202	0730	1
2	Karthik Iyer	AI101	0930	1
3	Sneha Reddy	AI303	1100	3
4	Vikram Sharma	AI202	0730	2
5	Ananya Gupta	AI404	0600	1
6	Rahul Verma	AI303	1100	1
7	Priya Nair	AI404	0600	2

19.1 Recursive Division (Divide Phase)

The array of 8 records is recursively divided until each sub-array holds a single element:

Original Array (indices 0-7):
[Arjun, Divya, Karthik, Sneha, Vikram, Ananya, Rahul, Priya]

Level 0: [Arjun,Divya,Karthik,Sneha,Vikram,Ananya,Rahul,Priya]

+-----+-----+

v v

Level 1: [Arjun,Divya,Karthik,Sneha] [Vikram,Ananya,Rahul,Priya]

+-----+-----+

v v v v

Level 2: [Arjun,Divya][Karthik,Sneha] [Vikram,Ananya][Rahul,Priya]

+-----+-----+

v v v v

Level 3: [Arjun][Divya][Karthik][Sneha] [Vikram][Ananya][Rahul][Priya]

(single-element sub-arrays -- base case reached)

19.2 Merge Operations (Combine Phase)

The single-element sub-arrays are now merged back together, level by level, comparing departure time first and priority second:

Merge Step 1 — Level 3 to Level 2

Left Sub-array	Right Sub-array	Comparison	Merged Result
[Arjun, 0930, P2]	[Divya, 0730, P1]	0730 < 0930 --> Divya first	[Divya(0730,P1), Arjun(0930,P2)]
[Karthik, 0930, P1]	[Sneha, 1100, P3]	0930 < 1100 --> Karthik first	[Karthik(0930,P1), Sneha(1100,P3)]
[Vikram, 0730, P2]	[Ananya, 0600, P1]	0600 < 0730 --> Ananya first	[Ananya(0600,P1), Vikram(0730,P2)]
[Rahul, 1100, P1]	[Priya, 0600, P2]	0600 < 1100 --> Priya first	[Priya(0600,P2), Rahul(1100,P1)]

Merge Step 2 — Level 2 to Level 1

Left group: merging [Divya(0730,P1), Arjun(0930,P2)] with [Karthik(0930,P1), Sneha(1100,P3)]

- Compare Divya(0730) vs Karthik(0930): 0730 < 0930 → take **Divya**.
- Compare Arjun(0930,P2) vs Karthik(0930,P1): same time, P2 > P1 → take **Karthik**.
- Compare Arjun(0930,P2) vs Sneha(1100,P3): 0930 < 1100 → take **Arjun**.
- Right list exhausted-side remainder → append **Sneha**.
- **Result:** [Divya(0730,P1), Karthik(0930,P1), Arjun(0930,P2), Sneha(1100,P3)]

Right group: merging [Ananya(0600,P1), Vikram(0730,P2)] with [Priya(0600,P2), Rahul(1100,P1)]

- Compare Ananya(0600,P1) vs Priya(0600,P2): same time, P1 < P2 → take **Ananya**.
- Compare Vikram(0730,P2) vs Priya(0600,P2): 0730 > 0600 → take **Priya**.
- Compare Vikram(0730,P2) vs Rahul(1100,P1): 0730 < 1100 → take **Vikram**.
- Remainder → append **Rahul**.
- **Result:** [Ananya(0600,P1), Priya(0600,P2), Vikram(0730,P2), Rahul(1100,P1)]

Merge Step 3 — Level 1 to Level 0 (Final Merge)

Merging [Divya(0730,P1), Karthik(0930,P1), Arjun(0930,P2), Sneha(1100,P3)] with [Ananya(0600,P1), Priya(0600,P2), Vikram(0730,P2), Rahul(1100,P1)]

- Divya(0730) vs Ananya(0600): 0730 > 0600 → take **Ananya**.
- Divya(0730) vs Priya(0600): 0730 > 0600 → take **Priya**.
- Divya(0730,P1) vs Vikram(0730,P2): same time, P1 < P2 → take **Divya**.
- Karthik(0930) vs Vikram(0730): 0930 > 0730 → take **Vikram**.
- Karthik(0930) vs Rahul(1100): 0930 < 1100 → take **Karthik**.
- Arjun(0930) vs Rahul(1100): 0930 < 1100 → take **Arjun**.

- Sneha(1100,P3) vs Rahul(1100,P1): same time, $P3 > P1 \rightarrow$ take **Rahul**.
- Remainder $\rightarrow$ append **Sneha**.

19.3 Final Sorted Output

Order	Passenger Name	Flight No.	Departure Time	Priority
1	Ananya Gupta	AI404	0600	1
2	Priya Nair	AI404	0600	2
3	Divya Menon	AI202	0730	1
4	Vikram Sharma	AI202	0730	2
5	Karthik Iyer	AI101	0930	1
6	Arjun Rao	AI101	0930	2
7	Rahul Verma	AI303	1100	1
8	Sneha Reddy	AI303	1100	3

The result confirms that bookings are correctly grouped by **departure time** and, within each group, ordered by **priority**, verifying both the correctness and the stability of Merge Sort.

20. Complexity Analysis

Case	Time Complexity	Space Complexity	Recursion Depth
Best Case	$O(n \log n)$	$O(n)$	$\log_2(n)$
Average Case	$O(n \log n)$	$O(n)$	$\log_2(n)$
Worst Case	$O(n \log n)$	$O(n)$	$\log_2(n)$

Why Merge Sort always takes $O(n \log n)$: Merge Sort divides the array into two halves at every recursive level, producing a recursion tree of height $\log_2(n)$. At every level, all sub-arrays together contain n elements, and merging them takes $O(n)$ time. Multiplying the $O(n)$ work per level by the $\log_2(n)$ levels gives a total time complexity of $O(n \log n)$ — and, unlike Quick Sort, this bound holds even in the worst case, because the split is always balanced regardless of the input data's initial arrangement.

21. Stability Analysis

A sorting algorithm is **stable** if two records with equal key values retain their original relative order after sorting. In the merge() procedure above, the condition `compare(left[j], right[j]) <= 0` always favours the

element from the **left** sub-array whenever the keys are equal. Since elements from the left half originally appeared before elements from the right half, this rule guarantees that equal-priority passengers with the same departure time are never reordered arbitrarily.

Stability is essential for an airline reservation system: passengers with the same departure time and same priority level (for example, two Economy-class passengers on the same flight) must appear in the boarding/booking list in the **order they were originally booked**, not in an unpredictable order.

22. Space Complexity

Merge Sort requires **$O(n)$ auxiliary space** because the merge() procedure creates temporary arrays to hold the left and right halves during each merge operation. This is a trade-off against in-place algorithms such as Insertion Sort or Heap Sort, but the trade-off is generally acceptable for airline systems, which typically run on servers with sufficient memory to sort even very large batches of booking records.

23. Applications

Domain	Use of Merge Sort
Airline Reservation Systems	Sorting bookings by departure time and passenger priority for boarding lists and itineraries.
Railway Reservation	Sorting waitlisted passengers and berth allocations by booking time and quota priority.
Banking	Sorting transaction records chronologically for statements and audit trails.
Hospital Management	Sorting patient records by admission time and case severity/priority.
Database Systems	Used internally in ORDER BY operations and index construction.
Cloud Computing	Sorting large distributed datasets across multiple nodes (distributed merge sort).
External Sorting	Sorting datasets too large to fit in main memory, using disk-based merge passes.
Big Data	Core sorting step in the shuffle phase of the MapReduce programming model.
Operating Systems	Sorting process/job queues by arrival time or scheduling priority.

24. Advantages

1. Guarantees **$O(n \log n)$** time complexity in the best, average, and worst cases.
2. **Stable** algorithm — preserves the relative order of equal keys.

3. Performs efficiently on **large datasets**, unlike quadratic-time algorithms.
4. Well suited for **external sorting** of data that does not fit in memory.
5. Predictable performance — not affected by the initial arrangement of data.
6. Works efficiently on **linked lists** as well as arrays.
7. Easily **parallelisable**, since the two halves can be sorted independently.
8. Simple recursive structure that is easy to implement and verify.
9. Suitable for **multi-key sorting** (e.g., departure time and priority) using a composite comparator.
10. Forms the basis of many production-grade sorting utilities and database engines.

25. Limitations

1. Requires **$O(n)$ extra memory**, unlike in-place algorithms such as Heap Sort.
2. Slightly slower than Quick Sort in practice for small arrays due to recursive overhead.
3. Recursive calls consume additional **stack space**, proportional to $\log(n)$.
4. Not adaptive — does not take advantage of any existing partial order in the input.
5. Copying elements between temporary arrays increases constant-factor overhead.
6. Less cache-friendly than in-place algorithms because of non-contiguous memory access during merging.
7. For very small datasets, the overhead of recursion can outweigh its benefits compared to Insertion Sort.
8. Implementation complexity is higher than simple algorithms like Bubble Sort or Selection Sort.

26. Comparison with Other Sorting Algorithms

Algorithm	Time Complexity (Worst)	Space	Stable	In-place	Best Use Case	Efficiency	Recommendation
Bubble Sort	$O(n^2)$	$O(1)$	Yes	Yes	Teaching / tiny lists	Low	Not Recommended
Selection Sort	$O(n^2)$	$O(1)$	No	Yes	Small, memory-constrained data	Low	Not Recommended
Insertion Sort	$O(n^2)$	$O(1)$	Yes	Yes	Small or nearly-sorted data	Moderate	Situational
Quick Sort	$O(n^2)^*$	$O(\log n)$	No	Yes	General-purpose, average case	High (avg.)	Situational

Algorithm	Time Complexity (Worst)	Space	Stable	In-place	Best Use Case	Efficiency	Recommendation
Heap Sort	$O(n \log n)$	$O(1)$	No	Yes	Memory-constrained systems	High	Situational
Merge Sort	$O(n \log n)$	$O(n)$	Yes	No	Large, stable, real-time data	High	Recommended

**Quick Sort's worst case occurs with a poor pivot choice; its average case is $O(n \log n)$.*

For the airline reservation use case — where **stability** and **predictable worst-case performance** on large, frequently-updated datasets are essential — **Merge Sort** is the clear recommendation, despite its higher space usage.

27. Real World Implementation in Airline Reservation Systems

In a production airline reservation system, Merge Sort can be integrated as follows:

- Newly created bookings are appended to an in-memory or database-backed list.
- When a sorted boarding list, itinerary, or report is requested, the system runs Merge Sort on the current booking set using (**departure_time, priority**) as the composite key.
- For very large historical datasets that exceed available memory, an **external merge sort** variant reads and merges sorted chunks from disk.
- Because Merge Sort is stable, repeated re-sorting after incremental updates does not disturb the relative order of passengers who were already correctly ordered.
- The divide step can be distributed across multiple servers in a cloud environment, allowing the system to scale horizontally during peak booking periods.

28. Sample Input

The unsorted input used for this case study (also shown in Section 19):

Passenger Name	Flight No.	Departure Time	Priority
Arjun Rao	AI101	0930	2
Divya Menon	AI202	0730	1
Karthik Iyer	AI101	0930	1
Sneha Reddy	AI303	1100	3
Vikram Sharma	AI202	0730	2

Passenger Name	Flight No.	Departure Time	Priority
Ananya Gupta	AI404	0600	1
Rahul Verma	AI303	1100	1
Priya Nair	AI404	0600	2

29. Sample Output

Console output produced by running the Python program in Section 17:

```
Before Sorting:
Arjun Rao      | AI101 | 0930 | P2
Divya Menon    | AI202 | 0730 | P1
Karthik Iyer   | AI101 | 0930 | P1
Sneha Reddy    | AI303 | 1100 | P3
Vikram Sharma  | AI202 | 0730 | P2
Ananya Gupta   | AI404 | 0600 | P1
Rahul Verma    | AI303 | 1100 | P1
Priya Nair     | AI404 | 0600 | P2

After Sorting (by Departure Time, then Priority):
Ananya Gupta   | AI404 | 0600 | P1
Priya Nair     | AI404 | 0600 | P2
Divya Menon    | AI202 | 0730 | P1
Vikram Sharma  | AI202 | 0730 | P2
Karthik Iyer   | AI101 | 0930 | P1
Arjun Rao      | AI101 | 0930 | P2
Rahul Verma    | AI303 | 1100 | P1
Sneha Reddy    | AI303 | 1100 | P3
```

32. Conclusion

Airline reservation systems must sort large volumes of booking records reliably while accommodating continuous real-time updates. This case study analysed the **Merge Sort** algorithm as a solution to this problem, using **departure time** as the primary sorting key and **passenger priority** as the secondary key.

Through a Divide and Conquer approach, Merge Sort recursively splits the booking array and merges sorted halves to achieve a guaranteed **$O(n \log n)$** time complexity in the best, average, and worst cases — a considerable improvement over the quadratic-time algorithms used in many legacy systems. The manual dry run in Section 19 confirmed that the algorithm produces a correctly ordered and **stable** result, preserving the original order of equal-priority bookings.

While Merge Sort requires $O(n)$ additional memory, this cost is acceptable given the performance, predictability, and stability it offers. Considering all these factors, **Merge Sort is the most suitable algorithm** for building an efficient, scalable, and fair airline reservation sorting system.

33. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] GeeksforGeeks, "Merge Sort," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/merge-sort/>
- [3] Programiz, "Merge Sort Algorithm," Programiz. [Online]. Available: <https://www.programiz.com/dsa/merge-sort>
- [4] Python Software Foundation, "Python 3 Documentation," Python.org. [Online]. Available: <https://docs.python.org/3/>
- [5] NPTEL, "Design and Analysis of Algorithms - Lecture Notes," NPTEL, Indian Institute of Technology. [Online]. Available: <https://nptel.ac.in/>